Gramática LL(1)

Compilador de instrucciones de C

Compiladores, Grupo 5 · Equipo 3
Facultad de Ingeniería - UNAM

Semestre 2027-1

1. Convenciones

- Los *no terminales* se escriben en cursiva y los `terminales` en letra monoespaciada, mientras que el símbolo inicial es *Program*.
- Los terminales `id`, `int_const`, `float_const`, `char_const` y `string_lit` representan clases de tokens del analizador léxico, mientras que todos los demás terminales (palabras clave, operadores y puntuación) coinciden con su lexema. `$` marca el fin de la entrada.
- Cuando un no terminal aparece más de una vez en una producción, las apariciones se distinguen con subíndices: en $AddTail \rightarrow AddOp\ MulExpr\ AddTail_1$, $AddTail$ es el padre y $AddTail_1$ el hijo, esto es similar a la sintaxis utilizada en la clase pero, en lugar de usar comillas sencillas para determiar una producción, utilizamos subíndice.

  Anteriormente era: $T \rightarrow T'$, donde T es el padre y T' es el hijo.

Tabla 1: Correspondencia entre tokens del lexer y terminales de la gramática

Token (`TokenType`)	**Terminal**	**Ejemplos**
`Keyword`, `Operator`, `Punctuation`	el propio lexema	`int`, `+=`, `{`
`Identifier`	`id`	`x`, `total`
`Constant` entera	`int_const`	`10`, `0755`
`Constant` con `.`, `e` o `E`	`float_const`	`3.14`, `2.5e-4`, `1.5f`
`Constant` entre `'`	`char_const`	`'A'`, `'\n'`
`Constant` entre `"`	`string_lit`	`"total =%d\n"`
`EndOfFile`	`$`	fin de la entrada

2. Gramática LL(1)

La gramática tiene 51 no terminales y 110 producciones (estas 110, se cuentan considerando que una producción tiene una o más producciones), esta se obtuvo a partir de una gramática de C convencional eliminando la recursión por la izquierda en las expresiones (un nivel por cada precedencia, del operador de menor precedencia al de mayor) y factorizando por la izquierda (left factoring) las producciones que compartían prefijo (*Item*, *Primary*).

Tabla 2: Producciones numeradas

(1)	*Program*	→	*ItemList*
(2)	*ItemList*	→	*Item ItemList*
(3)		|	ε
(4)	*Item*	→	*Type* `id` *ItemTail*
(5)		|	*Stmt*
(6)	*ItemTail*	→	`(` *ParamList* `)` *Block*
(7)		|	*VarTail*
(8)	*Type*	→	`int`
(9)		|	`float`
(10)		|	`double`
(11)		|	`char`
(12)		|	`void`
(13)	*ParamList*	→	*Param ParamMore*
(14)		|	ε
(15)	*ParamMore*	→	`,` *Param ParamMore*
(16)		|	ε
(17)	*Param*	→	*Type* `id`
(18)	*VarTail*	→	*Init VarMore* `;`
(19)	*VarMore*	→	`,` `id` *Init VarMore*
(20)		|	ε
(21)	*Init*	→	`=` *Expr*
(22)		|	ε
(23)	*Block*	→	`{` *BlockItems* `}`
(24)	*BlockItems*	→	*BlockItem BlockItems*
(25)		|	ε
(26)	*BlockItem*	→	*Type* `id` *VarTail*
(27)		|	*Stmt*
(28)	*Stmt*	→	*ExprStmt*
(29)		|	*Block*
(30)		|	*IfStmt*
(31)		|	*WhileStmt*
(32)		|	*DoStmt*
(33)		|	*ForStmt*
(34)		|	*ReturnStmt*
(35)		|	*IoStmt*
(36)		|	`break` `;`
(37)		|	`continue` `;`
(38)	*ExprStmt*	→	*Expr* `;`
(39)		|	`;`
(40)	*IfStmt*	→	`if` `(` *Expr* `)` *Stmt ElsePart*
(41)	*ElsePart*	→	`else` *Stmt*
(42)		|	ε
(43)	*WhileStmt*	→	`while` `(` *Expr* `)` *Stmt*
(44)	*DoStmt*	→	`do` *Stmt* `while` `(` *Expr* `)` `;`

(45) *ForStmt* → `for` `(` *ForInit* *OptExpr* `;` *OptExpr* `)` *Stmt*
(46) *ForInit* → *Type* `id` *VarTail*
(47) | *ExprStmt*
(48) *OptExpr* → *Expr*
(49) | ε
(50) *ReturnStmt* → `return` *OptExpr* `;`
(51) *IoStmt* → *IoFunc* `(` *ArgList* `)` `;`
(52) *IoFunc* → `printf`
(53) | `print`
(54) | `scanf`
(55) *ArgList* → *Expr* *ArgMore*
(56) | ε
(57) *ArgMore* → `,` *Expr* *ArgMore*
(58) | ε
(59) *Expr* → *OrExpr* *AssignTail*
(60) *AssignTail* → *AssignOp* *Expr*
(61) | ε
(62) *AssignOp* → `=`
(63) | `+=`
(64) | `-=`
(65) | `*=`
(66) | `/=`
(67) | `%=`
(68) *OrExpr* → *AndExpr* *OrTail*
(69) *OrTail* → `||` *AndExpr* *OrTail*
(70) | ε
(71) *AndExpr* → *EqExpr* *AndTail*
(72) *AndTail* → `&&` *EqExpr* *AndTail*
(73) | ε
(74) *EqExpr* → *RelExpr* *EqTail*
(75) *EqTail* → *EqOp* *RelExpr* *EqTail*
(76) | ε
(77) *EqOp* → `==`
(78) | `!=`
(79) *RelExpr* → *AddExpr* *RelTail*
(80) *RelTail* → *RelOp* *AddExpr* *RelTail*
(81) | ε
(82) *RelOp* → `<`
(83) | `>`
(84) | `<=`
(85) | `>=`
(86) *AddExpr* → *MulExpr* *AddTail*
(87) *AddTail* → *AddOp* *MulExpr* *AddTail*
(88) | ε
(89) *AddOp* → `+`
(90) | `-`
(91) *MulExpr* → *Unary* *MulTail*
(92) *MulTail* → *MulOp* *Unary* *MulTail*
(93) | ε
(94) *MulOp* → `*`
(95) | `/`
(96) | `%`
(97) *Unary* → *UnaryOp* *Unary*

(98)		|	*Primary*
(99)	*UnaryOp*	→	`-`
(100)		|	`+`
(101)		|	`!`
(102)		|	`&`
(103)	*Primary*	→	`id` *CallTail*
(104)		|	`int_const`
(105)		|	`float_const`
(106)		|	`char_const`
(107)		|	`string_lit`
(108)		|	`(` *Expr* `)`
(109)	*CallTail*	→	`(` *ArgList* `)`
(110)		|	ε

2.1. Condición LL(1) y el *else* colgante

Una gramática es LL(1) si para cada par de alternativas $A \to \alpha \mid \beta$ se cumple

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing \qquad \text{y} \qquad \beta \Rightarrow^* \varepsilon \implies \text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \varnothing,$$

es decir, si ninguna celda $M[A, a]$ de la tabla predictiva recibe dos producciones, el programa calcula FIRST, FOLLOW y la tabla a partir de la gramática (`parser -grammar`), y la única celda con dos producciones es $M[$*ElsePart*, `else`$]$, con (41) y (42). Esto se debe a que `else` $\in$ FIRST(`else` *Stmt*) y también `else` $\in$ FOLLOW(*ElsePart*). Es la ambigüedad del *else* colgante de C: en `if (a) if (b) x; else y;` el `else` podría pertenecer a cualquiera de los dos `if`. El conflicto se resuelve conservando (41) *ElsePart* → `else` *Stmt*, lo que asocia el `else` con el `if` más cercano, tal como establece el estándar de C.

3. Traducción dirigida por la sintaxis (SDT)

La traducción se especifica como una *definición dirigida por la sintaxis* (SDD): cada símbolo tiene atributos y cada producción tiene reglas semánticas que los calculan, todos los atributos son sintetizados o heredados del padre o de hermanos a la izquierda, por lo que la definición es **L-atribuida**, es por eso se evalúa con un único recorrido en profundidad, de izquierda a derecha, sobre el árbol de derivación, las reglas de cada producción se ejecutan en el orden en que aparecen, y una regla que usa un atributo de un hijo se ejecuta después de visitar a ese hijo.

3.1. Tipos

- Tipos base: `void`, `char`, `int`, `float`, `double`. Apuntadores: `char*` (literales de cadena) y `T*` (resultado de `&`x).
- **Numéricos**: `char`, `int`, `float`, `double`. **Enteros**: `char`, `int`. **Flotantes**: `float`, `double`. **Escalares**: numéricos y apuntadores.
- Rango: `char` < `int` < `float` < `double`. Promoción: **prom**(`char`) = `int` y **prom**(t) = t en otro caso.
- `err` es un tipo interno que se asigna a una expresión que ya produjo un error, toda función auxiliar que recibe `err` lo propaga sin reportar nada nuevo, para no repetir el mismo error en todos los niveles del árbol.
- $\perp$ significa "no existe": un tok que no es un único token, o un ret fuera de cualquier función.

3.2. Atributos

Tabla 3: Atributos de la SDD (S = sintetizado, H = heredado)

Símbolos	Atributo	Clase	Significado
`id` y demás terminales	lex, lín	léxico	Lexema y línea del token (los da el lexer).
Type	t	S	Tipo escrito en la declaración.
ItemTail, *VarTail*, *Init*	t, nombre, lín	H	Tipo declarado, nombre y línea del identificador que se declara.
VarMore	t	H	Tipo declarado, compartido por todos los declaradores de la lista.
Param	t, nombre, lín	S	Tipo, nombre y línea de un parámetro.
ParamList, *ParamMore*	ps	S	Lista de parámetros $\langle t, \text{nombre}, \text{línea} \rangle$.
Block	nuevo	H	Indica si el bloque abre un ámbito nuevo (falso solo en el cuerpo de una función).
ItemList, *Item*, *Block*, *BlockItems*, *BlockItem*, *Stmt*, *IfStmt*, *ElsePart*, *WhileStmt*, *DoStmt*, *ForStmt*, *ReturnStmt*	loop	H	Número de ciclos que encierran la sentencia (para `break`/`continue`).
(los mismos)	ret	H	Tipo de retorno de la función que encierra la sentencia, o $\perp$ fuera de funciones.
Expr, *OrExpr*, *AndExpr*, *EqExpr*, *RelExpr*, *AddExpr*, *MulExpr*, *Unary*, *Primary*	tipo	S	Tipo de la expresión.
(los mismos)	tok	S	El token, si la expresión consiste en un único token (una variable o un literal), o $\perp$. Sirve para saber si algo es asignable y si el formato de `printf` es un literal.
AssignTail, *OrTail*, *AndTail*, *EqTail*, *RelTail*, *AddTail*, *MulTail*	her	H	Tipo del operando izquierdo acumulado.
(los mismos)	sin	S	Tipo final de la cadena de operaciones.
(los mismos)	vacío	S	verdadero si la cola derivó ε.
AssignTail	izq	H	tok del lado izquierdo de la asignación.
AssignOp, *EqOp*, *RelOp*, *AddOp*, *MulOp*, *UnaryOp*, *IoFunc*	op	S	Lexema del operador o de la función de E/S.
OptExpr	tipo, vacío	S	Tipo de la expresión opcional y si está ausente.
ArgList, *ArgMore*, *CallTail*	args	S	Lista de argumentos $\langle \text{tipo}, \text{tok} \rangle$.
CallTail	vacío	S	falso si hay llamada a función.

Convención de copia. Los atributos heredados loop y ret pasan sin cambios del padre a cada hijo que los tiene, salvo en las producciones donde la tabla indica otro valor: (1), (6), (43), (44) y (45), por brevedad, esas copias no se escriben en la tabla.

Tabla de símbolos. Es una pila de ámbitos: uno global, uno por función (parámetros y cuerpo), uno por bloque y uno por cada `for`, cada entrada guarda la clase (var o función), el tipo, los tipos de los parámetros y la línea de declaración.

3.3. Funciones auxiliares

En todas las funciones, **error**(l, ...) registra un error en la línea l y **aviso**(l, ...) registra una advertencia. El análisis no se detiene en el primer error.

abrirÁmbito(), cerrarÁmbito()
Apilan y desapilan un ámbito de la tabla de símbolos.

buscar(n), **buscarLocal**(n)
Buscan n del ámbito más interno al global, o solo en el ámbito actual. Si no lo encuentran, devuelven $\perp$.

declarar(n, e, l)
si buscarLocal(n) $\neq \perp$ **entonces error**(l, redeclaración de n) **si no** inserta e en el ámbito actual. *(R1)*

declararVar(t, n, l)
si $t =$ `void` **entonces error**(l, variable `void`); $t =$ `err`. *(R2)*
Después: **declarar**(n, $\langle$var, $t\rangle$, l).

usarVariable(`id`)
$s =$ **buscar**(`id`.lex). **si** $s = \perp$ **entonces error**(no declarada) y devuelve `err`. *(R3)*
si s es función **entonces error**(función usada como valor) y devuelve `err`. **si no** devuelve s.tipo.

llamada(`id`, *args*)
$s =$ **buscar**(`id`.lex). **si** $s = \perp$ o s no es función **entonces error** y devuelve `err`. **si** $|args| \neq |s.\text{params}|$ **entonces error**(número de argumentos). **si no**, para cada i: **asignable**($s.\text{params}_i$, $args_i$.tipo, `id`.lín). Devuelve s.tipo. *(R10)*

asignable(d, o, l)
Regla para inicializaciones, asignaciones, argumentos y `return` (d = destino, o = origen). *(R4, W1)*

- **si** $d =$ `err` o $o =$ `err`: nada.
- **si** $o =$ `void`: **error**(un valor `void` no se puede usar).
- **si** d o o es apuntador: **si** $d \neq o$ **entonces error**(tipos incompatibles).
- **si** o es flotante y d es entero: **aviso**(se pierde la parte fraccionaria).
- En otro caso (numéricos) la conversión implícita es válida, como en C.

opBinaria($\circ$, a, b, l)
Tipo del resultado de $a \circ b$:

- **si** $a =$ `err` o $b =$ `err`: `err`.
- $\circ \in \{$`||`, `&&`$\}$: `int` si a y b son escalares. *(R8)*
- $\circ \in \{$`==`, `!=`, `<`, `>`, `<=`, `>=`$\}$: `int` si ambos son numéricos o son el mismo tipo apuntador. *(R8)*
- $\circ =$ `%`: `int` si ambos son enteros; si no, **error**(`%` requiere enteros). *(R7)*
- $\circ \in \{$`+`, `-`, `*`, `/`$\}$: si ambos son numéricos, **prom**(el de mayor rango). *(R6)*
- En otro caso: **error**(l, operandos inválidos) y devuelve `err`.

opUnaria($\circ$, t, ***tok***, l)
(R9)

- **si** $t =$ `err`: `err`.
- $\circ =$ `&`: *tok* debe ser un `id` y t no debe ser apuntador; el resultado es t`*`.
- $\circ =$ `!`: `int` si t es escalar.
- $\circ \in \{$`-`, `+`$\}$: **prom**(t) si t es numérico.
- En otro caso: **error**(l, operando inválido) y devuelve `err`.

condición(t, l, ***dónde***)
si $t \neq$ `err` y t no es escalar **entonces error**(l, la condición de *dónde* debe ser escalar). *(R11)*

revisarFormato(f, *args*, l)

Para $f \in \{$`printf`, `scanf`$\}$. *(R14)*

- **si** *args* $= [\,]$: **error**(falta el formato).
- **si** $args_1$.tok no es un `string_lit`: **error**(el formato debe ser un literal) y termina.
- S = especificadores del literal, de la forma `%[-+ #0]*[0-9]*(.[0-9]*)?l?c`, con $c \in$ `diuxXfFeEgGcs`. `%%` no cuenta; cualquier otro `%` produce **error**(especificador desconocido).
- **si** $|S| \neq |args| - 1$: **error**(número de especificadores y argumentos distinto).
- Para cada par $(S_i, args_{i+1})$ se revisa el tipo según la tabla 4. En `scanf`, un argumento que no es apuntador produce **error**(se espera una dirección `&x`).

revisarPrint(*args*, l)

si *args* $= [\,]$ **entonces** **error**(`print` necesita al menos un argumento). Cada argumento de tipo `void` produce **error**. *(R15)*

Tabla 4: Tipos aceptados por cada especificador de formato

Especificador	`printf`	`scanf`
`%d` `%i` `%u` `%x` `%X`	`int` o `char`	`int*`
`%c`	`char` o `int`	`char*`
`%f` `%e` `%g` (y mayúsculas)	`float` o `double`	`float*`
`%lf` `%le` `%lg`	`float` o `double`	`double*`
`%s`	`char*`	`char*`

3.4. Definición dirigida por la sintaxis completa

En las reglas, **lín**(X) es la línea del primer token derivado de X, las producciones marcadas con — no tienen reglas propias, aparte de la convención de copia de loop y ret.

Tabla 5: Producciones y reglas semánticas

#	Producción	Reglas semánticas
Programa		
(1)	*Program* → *ItemList*	**abrirÁmbito**() *(ámbito global)* *ItemList*.loop = 0; *ItemList*.ret = ⊥
(2)	*ItemList* → *Item* $ItemList_1$	—
(3)	*ItemList* → ε	—
Declaraciones		
(4)	*Item* → *Type* `id` *ItemTail*	*ItemTail*.t = *Type*.t *ItemTail*.nombre = `id`.lex; *ItemTail*.lín = `id`.lín
(5)	*Item* → *Stmt*	—
(6)	*ItemTail* → `(` *ParamList* `)` *Block*	**declarar**(*ItemTail*.nombre, ⟨función, *ItemTail*.t, **tipos**(*ParamList*.ps)⟩, *ItemTail*.lín) **abrirÁmbito**() **para cada** $\langle t, n, l\rangle \in$ *ParamList*.ps: **si** $t =$ `void` **entonces** **error**(l, parámetro `void`); $t =$ `err` **declarar**(n, ⟨var, t⟩, l) *(nombre repetido ⇒ parámetro duplicado)* *Block*.nuevo = falso; *Block*.ret = *ItemTail*.t; *Block*.loop = 0 **cerrarÁmbito**() *(después de visitar Block)*
(7)	*ItemTail* → *VarTail*	*VarTail*.t = *ItemTail*.t; *VarTail*.nombre = *ItemTail*.nombre; *VarTail*.lín = *ItemTail*.lín

#	Producción	Reglas semánticas
(8)	*Type* → `int`	*Type*.t = `int`
(9)	*Type* → `float`	*Type*.t = `float`
(10)	*Type* → `double`	*Type*.t = `double`
(11)	*Type* → `char`	*Type*.t = `char`
(12)	*Type* → `void`	*Type*.t = `void`
(13)	*ParamList* → *Param ParamMore*	*ParamList*.ps = [⟨*Param*.t, *Param*.nombre, *Param*.lín⟩] ++ *ParamMore*.ps
(14)	*ParamList* → ε	*ParamList*.ps = []
(15)	*ParamMore* → `,` *Param* $ParamMore_1$	*ParamMore*.ps = [⟨*Param*.t, *Param*.nombre, *Param*.lín⟩] ++ $ParamMore_1$.ps
(16)	*ParamMore* → ε	*ParamMore*.ps = []
(17)	*Param* → *Type* `id`	*Param*.t = *Type*.t; *Param*.nombre = `id`.lex; *Param*.lín = `id`.lín
(18)	*VarTail* → *Init VarMore* `;`	*Init*.t = *VarTail*.t; *Init*.nombre = *VarTail*.nombre; *Init*.lín = *VarTail*.lín *VarMore*.t = *VarTail*.t
(19)	*VarMore* → `,` `id` *Init* $VarMore_1$	*Init*.t = *VarMore*.t; *Init*.nombre = `id`.lex; *Init*.lín = `id`.lín $VarMore_1$.t = *VarMore*.t
(20)	*VarMore* → ε	—
(21)	*Init* → `=` *Expr*	**si** *Init*.t ≠ `void` **entonces** **asignable**(*Init*.t, *Expr*.tipo, *Init*.lín) **declararVar**(*Init*.t, *Init*.nombre, *Init*.lín) *(después de evaluar Expr)*
(22)	*Init* → ε	**declararVar**(*Init*.t, *Init*.nombre, *Init*.lín)
	Sentencias	
(23)	*Block* → `{` *BlockItems* `}`	**si** *Block*.nuevo **entonces** **abrirÁmbito**() *(antes de BlockItems)* **si** *Block*.nuevo **entonces** **cerrarÁmbito**() *(después de BlockItems)*
(24)	*BlockItems* → *BlockItem* $BlockItems_1$	—
(25)	*BlockItems* → ε	—
(26)	*BlockItem* → *Type* `id` *VarTail*	*VarTail*.t = *Type*.t; *VarTail*.nombre = `id`.lex; *VarTail*.lín = `id`.lín
(27)	*BlockItem* → *Stmt*	—
(28)	*Stmt* → *ExprStmt*	—
(29)	*Stmt* → *Block*	*Block*.nuevo = verdadero
(30)	*Stmt* → *IfStmt*	—
(31)	*Stmt* → *WhileStmt*	—
(32)	*Stmt* → *DoStmt*	—
(33)	*Stmt* → *ForStmt*	—
(34)	*Stmt* → *ReturnStmt*	—
(35)	*Stmt* → *IoStmt*	—
(36)	*Stmt* → `break` `;`	**si** *Stmt*.loop = 0 **entonces** **error**(`break`.lín, `break` fuera de un ciclo)
(37)	*Stmt* → `continue` `;`	**si** *Stmt*.loop = 0 **entonces** **error**(`continue`.lín, `continue` fuera de un ciclo)
(38)	*ExprStmt* → *Expr* `;`	*(se evalúa Expr y su tipo se descarta)*
(39)	*ExprStmt* → `;`	—
(40)	*IfStmt* → `if` `(` *Expr* `)` *Stmt ElsePart*	**condición**(*Expr*.tipo, **lín**(*Expr*), `if`)
(41)	*ElsePart* → `else` *Stmt*	—
(42)	*ElsePart* → ε	—

#	Producción	Reglas semánticas
(43)	*WhileStmt* → `while ( Expr )` *Stmt*	**condición**(*Expr*.tipo, **lín**(*Expr*), `while`) *Stmt*.loop = *WhileStmt*.loop + 1
(44)	*DoStmt* → `do` *Stmt* `while ( Expr ) ;`	*Stmt*.loop = *DoStmt*.loop + 1 **condición**(*Expr*.tipo, **lín**(*Expr*), `do-while`)
(45)	*ForStmt* → `for (` *ForInit* $OptExpr_1$ `;` $OptExpr_2$ `)` *Stmt*	**abrirÁmbito**() *(antes de ForInit: la variable de control es local al ciclo)* **si no** $OptExpr_1$.vacío **entonces condición**($OptExpr_1$.tipo, **lín**($OptExpr_1$), `for`) *Stmt*.loop = *ForStmt*.loop + 1 **cerrarÁmbito**() *(después de Stmt)*
(46)	*ForInit* → *Type* `id` *VarTail*	*VarTail*.t = *Type*.t; *VarTail*.nombre = `id`.lex; *VarTail*.lín = `id`.lín
(47)	*ForInit* → *ExprStmt*	—
(48)	*OptExpr* → *Expr*	*OptExpr*.tipo = *Expr*.tipo; *OptExpr*.vacío = falso
(49)	*OptExpr* → ε	*OptExpr*.tipo = `void`; *OptExpr*.vacío = verdadero
(50)	*ReturnStmt* → `return` *OptExpr* `;`	**si** *ReturnStmt*.ret = $\perp$ **entonces error**(l, `return` fuera de una función) **si no, si** *OptExpr*.vacío **y** *ReturnStmt*.ret $\neq$ `void` **entonces error**(l, `return` sin valor) **si no, si no** *OptExpr*.vacío **y** *ReturnStmt*.ret = `void` **entonces error**(l, `return` con valor en función `void`) **si no, si no** *OptExpr*.vacío **entonces asignable**(*ReturnStmt*.ret, *OptExpr*.tipo, l) *donde* l = *`return`.lín*
(51)	*IoStmt* → *IoFunc* `(` *ArgList* `) ;`	**si** *IoFunc*.op = `print` **entonces revisarPrint**(*ArgList*.args, l) **si no revisarFormato**(*IoFunc*.op, *ArgList*.args, l) *donde* l = ***lín**(IoStmt)*
	Entrada/salida y argumentos	
(52)	*IoFunc* → `printf`	*IoFunc*.op = `printf`
(53)	*IoFunc* → `print`	*IoFunc*.op = `print`
(54)	*IoFunc* → `scanf`	*IoFunc*.op = `scanf`
(55)	*ArgList* → *Expr* *ArgMore*	*ArgList*.args = [⟨*Expr*.tipo, *Expr*.tok⟩] ++ *ArgMore*.args
(56)	*ArgList* → ε	*ArgList*.args = []
(57)	*ArgMore* → `,` *Expr* $ArgMore_1$	*ArgMore*.args = [⟨*Expr*.tipo, *Expr*.tok⟩] ++ $ArgMore_1$.args
(58)	*ArgMore* → ε	*ArgMore*.args = []
	Asignación	
(59)	*Expr* → *OrExpr* *AssignTail*	*AssignTail*.her = *OrExpr*.tipo; *AssignTail*.izq = *OrExpr*.tok *Expr*.tipo = *AssignTail*.sin *Expr*.tok = **si** *AssignTail*.vacío **entonces** *OrExpr*.tok **si no** $\perp$
(60)	*AssignTail* → *AssignOp* *Expr*	*AssignTail*.vacío = falso; l = **lín**(*AssignOp*) **si** *AssignTail*.izq no es un `id` **entonces error**(l, el lado izquierdo no es una variable); *AssignTail*.sin = `err` **si no, si** *AssignTail*.her = `err` **entonces** *AssignTail*.sin = `err` **si no:** v = **si** *AssignOp*.op = `=` **entonces** *Expr*.tipo **si no opBinaria**($\circ$, *AssignTail*.her, *Expr*.tipo, l) *($\circ$ es el operador sin el `=`: `+=` ⇒ `+`)* **si** $v \neq$ `err` **entonces asignable**(*AssignTail*.her, v, l); *AssignTail*.sin = **si** v = `err` **entonces** `err` **si no** *AssignTail*.her
(61)	*AssignTail* → ε	*AssignTail*.sin = *AssignTail*.her; *AssignTail*.vacío = verdadero
(62)	*AssignOp* → `=`	*AssignOp*.op = `=`
(63)	*AssignOp* → `+=`	*AssignOp*.op = `+=`
(64)	*AssignOp* → `-=`	*AssignOp*.op = `-=`
(65)	*AssignOp* → `*=`	*AssignOp*.op = `*=`

#	Producción	Reglas semánticas
(66)	$AssignOp \to$ `/=`	$AssignOp$.op = `/=`
(67)	$AssignOp \to$ `%=`	$AssignOp$.op = `%=`
Expresiones binarias		
(68)	$OrExpr \to AndExpr\ OrTail$	$OrTail$.her = $AndExpr$.tipo $OrExpr$.tipo = $OrTail$.sin $OrExpr$.tok = **si** $OrTail$.vacío **entonces** $AndExpr$.tok **si no** $\perp$
(69)	$OrTail \to$ `||` $AndExpr\ OrTail_1$	$OrTail_1$.her = **opBinaria**(`||`, $OrTail$.her, $AndExpr$.tipo, `||`.lín) $OrTail$.sin = $OrTail_1$.sin; $OrTail$.vacío = falso
(70)	$OrTail \to \varepsilon$	$OrTail$.sin = $OrTail$.her; $OrTail$.vacío = verdadero
(71)	$AndExpr \to EqExpr\ AndTail$	$AndTail$.her = $EqExpr$.tipo $AndExpr$.tipo = $AndTail$.sin $AndExpr$.tok = **si** $AndTail$.vacío **entonces** $EqExpr$.tok **si no** $\perp$
(72)	$AndTail \to$ `&&` $EqExpr\ AndTail_1$	$AndTail_1$.her = **opBinaria**(`&&`, $AndTail$.her, $EqExpr$.tipo, `&&`.lín) $AndTail$.sin = $AndTail_1$.sin; $AndTail$.vacío = falso
(73)	$AndTail \to \varepsilon$	$AndTail$.sin = $AndTail$.her; $AndTail$.vacío = verdadero
(74)	$EqExpr \to RelExpr\ EqTail$	$EqTail$.her = $RelExpr$.tipo $EqExpr$.tipo = $EqTail$.sin $EqExpr$.tok = **si** $EqTail$.vacío **entonces** $RelExpr$.tok **si no** $\perp$
(75)	$EqTail \to EqOp\ RelExpr\ EqTail_1$	$EqTail_1$.her = **opBinaria**($EqOp$.op, $EqTail$.her, $RelExpr$.tipo, **lín**($EqOp$)) $EqTail$.sin = $EqTail_1$.sin; $EqTail$.vacío = falso
(76)	$EqTail \to \varepsilon$	$EqTail$.sin = $EqTail$.her; $EqTail$.vacío = verdadero
(77)	$EqOp \to$ `==`	$EqOp$.op = `==`
(78)	$EqOp \to$ `!=`	$EqOp$.op = `!=`
(79)	$RelExpr \to AddExpr\ RelTail$	$RelTail$.her = $AddExpr$.tipo $RelExpr$.tipo = $RelTail$.sin $RelExpr$.tok = **si** $RelTail$.vacío **entonces** $AddExpr$.tok **si no** $\perp$
(80)	$RelTail \to RelOp\ AddExpr\ RelTail_1$	$RelTail_1$.her = **opBinaria**($RelOp$.op, $RelTail$.her, $AddExpr$.tipo, **lín**($RelOp$)) $RelTail$.sin = $RelTail_1$.sin; $RelTail$.vacío = falso
(81)	$RelTail \to \varepsilon$	$RelTail$.sin = $RelTail$.her; $RelTail$.vacío = verdadero
(82)	$RelOp \to$ `<`	$RelOp$.op = `<`
(83)	$RelOp \to$ `>`	$RelOp$.op = `>`
(84)	$RelOp \to$ `<=`	$RelOp$.op = `<=`
(85)	$RelOp \to$ `>=`	$RelOp$.op = `>=`
(86)	$AddExpr \to MulExpr\ AddTail$	$AddTail$.her = $MulExpr$.tipo $AddExpr$.tipo = $AddTail$.sin $AddExpr$.tok = **si** $AddTail$.vacío **entonces** $MulExpr$.tok **si no** $\perp$
(87)	$AddTail \to AddOp\ MulExpr\ AddTail_1$	$AddTail_1$.her = **opBinaria**($AddOp$.op, $AddTail$.her, $MulExpr$.tipo, **lín**($AddOp$)) $AddTail$.sin = $AddTail_1$.sin; $AddTail$.vacío = falso
(88)	$AddTail \to \varepsilon$	$AddTail$.sin = $AddTail$.her; $AddTail$.vacío = verdadero
(89)	$AddOp \to$ `+`	$AddOp$.op = `+`
(90)	$AddOp \to$ `-`	$AddOp$.op = `-`
(91)	$MulExpr \to Unary\ MulTail$	$MulTail$.her = $Unary$.tipo $MulExpr$.tipo = $MulTail$.sin $MulExpr$.tok = **si** $MulTail$.vacío **entonces** $Unary$.tok **si no** $\perp$
(92)	$MulTail \to MulOp\ Unary\ MulTail_1$	$MulTail_1$.her = **opBinaria**($MulOp$.op, $MulTail$.her, $Unary$.tipo, **lín**($MulOp$)) $MulTail$.sin = $MulTail_1$.sin; $MulTail$.vacío = falso
(93)	$MulTail \to \varepsilon$	$MulTail$.sin = $MulTail$.her; $MulTail$.vacío = verdadero
(94)	$MulOp \to$ `*`	$MulOp$.op = `*`
(95)	$MulOp \to$ `/`	$MulOp$.op = `/`

#	Producción	Reglas semánticas
(96)	*MulOp* → `%`	*MulOp*.op = `%`
Expresiones unarias y primarias		
(97)	*Unary* → *UnaryOp* $Unary_1$	*Unary*.tipo = **opUnaria**(*UnaryOp*.op, $Unary_1$.tipo, $Unary_1$.tok, **lín**(*UnaryOp*)) *Unary*.tok = ⊥
(98)	*Unary* → *Primary*	*Unary*.tipo = *Primary*.tipo; *Unary*.tok = *Primary*.tok
(99)	*UnaryOp* → `-`	*UnaryOp*.op = `-`
(100)	*UnaryOp* → `+`	*UnaryOp*.op = `+`
(101)	*UnaryOp* → `!`	*UnaryOp*.op = `!`
(102)	*UnaryOp* → `&`	*UnaryOp*.op = `&`
(103)	*Primary* → `id` *CallTail*	**si** *CallTail*.vacío **entonces** *Primary*.tipo = **usarVariable**(`id`); *Primary*.tok = `id` **si no** *Primary*.tipo = **llamada**(`id`, *CallTail*.args); *Primary*.tok = ⊥
(104)	*Primary* → `int_const`	*Primary*.tipo = `int`; *Primary*.tok = `int_const`
(105)	*Primary* → `float_const`	*Primary*.tipo = **si** el lexema termina en `f` o `F` **entonces** `float` **si no** `double` *Primary*.tok = `float_const`
(106)	*Primary* → `char_const`	*Primary*.tipo = `char`; *Primary*.tok = `char_const`
(107)	*Primary* → `string_lit`	*Primary*.tipo = `char*`; *Primary*.tok = `string_lit`
(108)	*Primary* → `(` *Expr* `)`	*Primary*.tipo = *Expr*.tipo; *Primary*.tok = ⊥
(109)	*CallTail* → `(` *ArgList* `)`	*CallTail*.args = *ArgList*.args; *CallTail*.vacío = falso
(110)	*CallTail* → ε	*CallTail*.args = [ ]; *CallTail*.vacío = verdadero

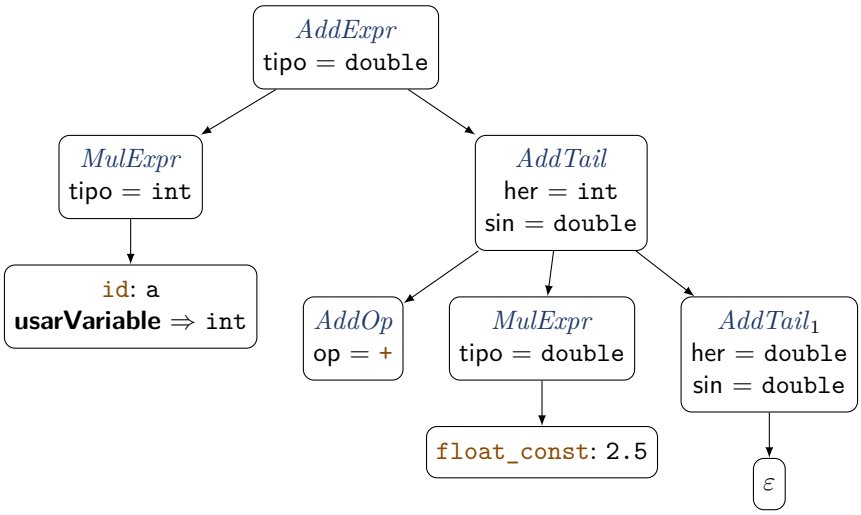
3.5. Resumen de las reglas semánticas

Regla	Producciones	Qué verifica	Ejemplo que la viola
R1	(6), (21), (22)	Ningún nombre se declara dos veces en el mismo ámbito.	`int x; int x;`
R2	(21), (22), (6)	Ninguna variable ni parámetro es `void`.	`void v;`
R3	(103)	Toda variable usada está declarada en un ámbito visible.	`y = 1;`
R4	(21), (50), (60), (103)	El valor es compatible con el destino.	`int n = "hola";`
R5	(60)	El lado izquierdo de una asignación es una variable.	`5 = n;`
R6	(87), (92)	`+ - * /` con operandos numéricos.	`"a" * 2`
R7	(92)	`%` con operandos enteros.	`2.5% 2`
R8	(69), (72), (75), (80)	Operandos lógicos escalares y operandos comparables.	`"a" < 3`
R9	(97)	Operandos válidos para `- + ! &`; `&` solo sobre variables.	`&(a + 1)`
R10	(103)	Llamadas a funciones declaradas, con el número y tipo de argumentos correctos.	`sum(1)`
R11	(40), (43), (44), (45)	Condiciones escalares.	`while (p())`, `p` `void`
R12	(36), (37)	`break` y `continue` solo dentro de ciclos.	`break;`
R13	(50)	`return` dentro de función y coherente con su tipo.	`return 1;` en `void`

Regla	Producciones	Qué verifica	Ejemplo que la viola
R14	(51)	Formato de `printf`/`scanf`: literal, cantidad y tipos.	`printf("%d", 2.5);`
R15	(51)	`print` con argumentos no `void`.	`print();`
W1	(21), (50), (60), (103)	Aviso (no error) al pasar de flotante a entero.	`int t = 3.7;`

3.6. Ejemplo: árbol anotado

Para la expresión `a + 2.5` con `a` declarada como `int`, el atributo heredado her lleva el tipo del operando izquierdo hacia la cola. El sintetizado sin sube el resultado de **opBinaria**(+, `int`, `double`) = `double`:



Por simplicidad, el árbol omite los niveles intermedios *MulExpr* ⇒ *Unary* ⇒ *Primary* y las colas vacías, que solo copian el tipo hacia arriba, el orden en que se calculan los atributos es: *MulExpr*.tipo (izquierdo), *AddTail*.her, *MulExpr*.tipo (derecho), $AddTail_1$.her, $AddTail_1$.sin, *AddTail*.sin y *AddExpr*.tipo, es un recorrido en profundidad de izquierda a derecha, como corresponde a una SDD L-atribuida.